

Numerically Stable Cholesky-QR on GPU via Mixed-Precision Randomized Preconditioning

James E. Garrison
Dept. of Mathematics
North Carolina State University
Raleigh, NC, USA
jegarri3@ncsu.edu

Chao Chen
Dept. of Mathematics
North Carolina State University
Raleigh, NC, USA
ccchen49@ncsu.edu

Ilse C. F. Ipsen
Dept. of Mathematics
North Carolina State University
Raleigh, NC, USA
ipsen@ncsu.edu

Abstract—Cholesky-QR is among the fastest algorithms for computing the thin QR factorization of tall-and-skinny matrices on GPUs, relying entirely on BLAS-3 operations. However, it is numerically unstable: forming the Gram matrix squares the condition number, causing breakdown when $\kappa_2(A) \gtrsim 10^8$. We present MRCQR (Mixed-Precision Randomized Cholesky-QR), a stable GPU algorithm that addresses this limitation. MRCQR uses a subsampled randomized trigonometric transform to construct a preconditioner R_s that reduces $\kappa_2(AR_s^{-1})$ to near unity with high probability, then applies Cholesky-QR in double precision to the preconditioned matrix. The key insight—supported by perturbation analysis—is that the preconditioner requires far less accuracy than the final result: single (FP32) precision suffices when $\kappa_2(A) \lesssim 10^8$, and half (FP16) when $\kappa_2(A) \lesssim 10^4$. MRCQR produces an explicit orthogonal factor $\hat{Q}$ satisfying $\|I - \hat{Q}^T \hat{Q}\|_2 = \mathcal{O}(u)$ ($u \approx 10^{-16}$, double-precision unit roundoff) for condition numbers up to 10^{16} , far beyond the 10^8 limit of `cho1qr2`. Experiments on an NVIDIA H100 GPU show that MRCQR (FP16) outperforms `rand-cho1qr` by 1.4–1.8 $\times$ across all tested column counts and is 1.8–13.5 $\times$ faster than `cuSOLVER geqrf`, while the FP16 sketch (used when $\kappa_2(A) \lesssim 10^4$) is 2 $\times$ cheaper than FP64 at no accuracy cost.

Index Terms—Mixed Precision, Tall-Skinny QR, Randomized Preconditioning, GPU Computing

I. INTRODUCTION

Given a matrix $A \in \mathbb{R}^{m \times n}$ with $m \gg n$ and $\text{rank}(A) = n$, we are interested in computing the thin QR factorization

$$A = QR, \quad Q^T Q = I_n,$$

where $Q \in \mathbb{R}^{m \times n}$ has orthonormal columns and $R \in \mathbb{R}^{n \times n}$ is upper triangular. In this paper we target double-precision output, though the same principle generalizes to other working precisions. Such computation is required for dimension reduction techniques [1], the solution of linear least squares problems [2], and inner loops of Krylov space methods [3], among others.

In several of these settings—particularly Krylov subspace methods and randomized matrix approximation— Q must be formed *explicitly* as a dense matrix, since it serves directly as an orthonormal basis for subsequent computations. The orthogonality of this explicit Q is critical: in Krylov subspace methods, for instance, loss of orthogonality among basis vectors causes spurious eigenvalues and premature stagnation

of convergence [3]. Moreover, many of these applications require the factorization to be performed *repeatedly*—the block Arnoldi process orthogonalizes a new block of vectors at each iteration [3], and randomized SVD via power iteration applies a QR factorization at each of its power steps [4]—so per-call cost accumulates directly. Together, these demands call for a QR routine that produces a numerically orthonormal explicit Q and is fast enough for repeated use on a GPU.

A. Background on Householder- and Cholesky-QR Methods

The performance of QR factorization on cache-based architectures has long been driven by maximizing BLAS-3 usage: the blocked `geqrf` algorithm casts the trailing matrix updates into Level-3 BLAS, while the panel factorization remains a BLAS-2 bottleneck [5, Section 5.2]. On modern GPUs, where the throughput gap between BLAS-3 and BLAS-2 operations is far more pronounced, this panel bottleneck becomes the dominant performance limiter, motivating algorithms that eliminate it entirely. In practice, GPU users typically rely on `cuSOLVER`’s `geqrf/orgqr` [6], which carry this same bottleneck onto the GPU. While numerically stable, this approach underutilizes GPU parallelism for tall-and-skinny matrices where $n \ll m$.

The Tall-Skinny QR (TSQR) algorithm [7] improves on this by organizing the computation as a tree of local Householder QR factorizations, achieving communication-optimality and numerical stability. However, TSQR has further practical limitations on GPU hardware: it is not exposed in vendor libraries such as `cuSOLVER`, and forming an explicit Q requires an additional backward pass through the tree, increasing data movement. In practice, TSQR is frequently slower than simpler alternatives on GPUs [8], and even authors of optimized GPU implementations do not recommend it for matrices with more than approximately 32 columns on Nvidia H100 GPU due to its limited amount of shared memory [9].

A communication-avoiding, more GPU-friendly approach is Cholesky-QR, which replaces Householder reflections with BLAS-3 matrix operations that map efficiently onto GPU architectures. We present the basic Cholesky-QR algorithm as Algorithm 1. It reduces the problem to just three vendor-library calls: a single matrix multiplication forming the (small) Gram matrix $A^T A$, followed by a small Cholesky factorization and

a triangular solve. For tall-and-skinny matrices with $n \ll m$, the dominant cost is the Gram matrix formation (step 1) and the triangular solve (step 3). The former is a compute-bound GEMM that maps well onto GPU tensor cores [10]. The triangular solve is formally BLAS-3, but for small n its sequential dependency structure makes it memory-bandwidth bound in practice, limiting GPU utilization. Despite this bottleneck, Cholesky-QR remains highly competitive when communication dominates arithmetic [11], a common regime in large-scale GPU computing.

Algorithm 1 Cholesky-QR (CholQR)

Input: $\mathbf{A} \in \mathbb{R}^{m \times n}$ with $m \gg n$ and $\text{rank}(\mathbf{A}) = n$

Output: $\mathbf{Q} \in \mathbb{R}^{m \times n}$ and upper triangular $\mathbf{R} \in \mathbb{R}^{n \times n}$

- 1: $\mathbf{G} \leftarrow \mathbf{A}^\top \mathbf{A}$ $\triangleright$ (Small) Gram matrix
 - 2: Compute $\mathbf{R}$ s.t. $\mathbf{G} = \mathbf{R}^\top \mathbf{R}$ $\triangleright$ Cholesky factorization
 - 3: $\mathbf{Q} \leftarrow \mathbf{A}\mathbf{R}^{-1}$ $\triangleright$ Triangular solve
-

In exact arithmetic, it is straightforward to verify that the output of Algorithm 1 $\mathbf{Q} = \mathbf{A}\mathbf{R}^{-1}$ satisfies $\mathbf{Q}^\top \mathbf{Q} = \mathbf{R}^{-\top} \mathbf{A}^\top \mathbf{A} \mathbf{R}^{-1} = \mathbf{R}^{-\top} \mathbf{R}^\top \mathbf{R} \mathbf{R}^{-1} = \mathbf{I}_n$, i.e., the columns of $\mathbf{Q}$ are exactly orthonormal. However, in floating-point arithmetic, forming $\mathbf{A}^\top \mathbf{A}$ squares the condition number of $\mathbf{A}$, and thus the computed $\hat{\mathbf{Q}}$ can deviate significantly from orthonormality by $\mathcal{O}(\kappa_2(\mathbf{A})^2 \mathbf{u})$ [12], where $\kappa_2(\mathbf{A})$ is the ratio between the largest and the smallest singular values of $\mathbf{A}$ and $\mathbf{u} = 2^{-53} \approx 1.1 \cdot 10^{-16}$ is the double-precision unit roundoff [13]. More critically, if $\kappa_2(\mathbf{A}) \gtrsim \mathbf{u}^{-1/2} \approx 10^8$, this squaring effect causes the computed Gram matrix to fail to be positive definite, so the Cholesky factorization breaks down entirely [14].

The standard remedy, CholQR2 [12], applies Algorithm 1 twice to correct the orthogonality error from $\mathcal{O}(\kappa_2(\mathbf{A})^2 \mathbf{u})$ to $\mathcal{O}(\mathbf{u})$, but it shares the same stability limit: both methods fail when $\kappa_2(\mathbf{A}) \gtrsim 10^8$, since the first pass requires the Gram matrix to be positive definite. Shifted CholQR3 [15] adds a stabilizing shift to the Gram matrix, extending stability to $\kappa_2(\mathbf{A}) \lesssim 10^{16}$, but at approximately 50% higher cost than CholQR2. Randomized preconditioning approaches [16]–[19] achieve the same stability range by replacing the extra passes in CholQR2 with an inexpensive sketch, at roughly the same cost as CholQR2. In particular, rand-cholQR [18] uses a two-level sparse-plus-dense multisketch and is the strongest prior randomized baseline. Our proposed method builds on these randomized approaches by additionally exploiting mixed precision in the sketch phase to reduce cost without sacrificing stability or accuracy. The idea of computing a randomized preconditioner in reduced precision has been explored for iterative solvers: Georgiou et al. [20] show GPU speedups for LSQR, and Carson and Daužickaitė [21] analyse mixed-precision sketching for GMRES-based iterative refinement. MRCQR targets a different setting—direct QR factorization with explicit $\mathbf{Q}$ formation—where the accuracy metric is orthogonality rather than iterative convergence, and the dominant cost after preconditioning is triangular solves and Gram matrix formation on the full tall-and-skinny matrix.

B. Our Approach and Contributions

Our proposed method, *Mixed Precision Randomized Preconditioned Cholesky-QR* (MRCQR), constructs an upper triangular preconditioner $\mathbf{R}_s$ such that $\mathbf{A}\mathbf{R}_s^{-1}$ is well-conditioned, then applies a single pass of Algorithm 1 to produce an explicit $\mathbf{Q}$ satisfying

$$\|\mathbf{I} - \hat{\mathbf{Q}}^\top \hat{\mathbf{Q}}\|_2 = \mathcal{O}(\mathbf{u}) \quad (1)$$

Specifically, $\mathbf{R}_s$ is the R -factor from the QR decomposition of $\Omega \mathbf{A} \in \mathbb{R}^{c \times n}$ with $n \leq c \ll m$, where Ω is a randomized sketching matrix [4]. This preconditioning idea was introduced for least-squares by Rokhlin and Tygert [22], [23] and has been applied to CholQR in prior work [16]–[19]; the novelty of MRCQR lies in exploiting mixed precision for the sketch.

Contribution 1: mixed-precision sketch. The main observation of this work is that $\mathbf{R}_s$ need not be computed in full precision when $\kappa_2(\mathbf{A})$ is modest, without sacrificing any final accuracy. Targeting double-precision orthogonality 10^{-16} in Eq. (1), our perturbation analysis in Theorem 1 establishes that:

- single precision suffices to construct $\mathbf{R}_s$ when $\kappa_2(\mathbf{A}) \lesssim 10^8$, and
- half precision suffices when $\kappa_2(\mathbf{A}) \lesssim 10^4$.

Given a coarse estimate of $\kappa_2(\mathbf{A})$, one can therefore select the lowest viable precision for the sketch, reducing both computation and memory traffic with no loss in final orthogonality. This observation is general and can be applied to improve the performance of other preconditioned CholQR methods.

Contribution 2: extended numerical stability. MRCQR achieves near-machine-precision orthogonality $\|\mathbf{I} - \hat{\mathbf{Q}}^\top \hat{\mathbf{Q}}\|_2 = \mathcal{O}(\mathbf{u})$ for $\kappa_2(\mathbf{A})$ up to 10^{16} —matching the stability range of CholQR3 [15] but without its 50% cost overhead, and far beyond the 10^8 limit shared by CholQR and CholQR2 [12]. MRCQR remains backward-stable across all tested condition numbers. Experiments sweeping $\kappa_2(\mathbf{A}) \in \{10^2, \dots, 10^{16}\}$ confirm the theoretical precision thresholds: the single-precision sketch maintains $\mathcal{O}(\mathbf{u})$ orthogonality for $\kappa_2(\mathbf{A}) \lesssim 10^8$ and the half-precision sketch for $\kappa_2(\mathbf{A}) \lesssim 10^4$.

Contribution 3: efficient GPU implementation. MRCQR is built entirely on vendor-optimized libraries (cuFFT, cuBLAS, cuSOLVER [6]), requiring no custom GPU kernels and making it immediately deployable on any CUDA device. Profiling on the NVIDIA H100 reveals that the two triangular solves (TRSM) dominate runtime (56–74% for $n = 16$ –128) while the sketch phase accounts for only 10–20%—the only component that changes with precision. Switching from double to half-precision sketch reduces sketch cost by $\approx 2\times$ at no accuracy cost, and MRCQR (FP16) outperforms rand-cholQR [18] by 1.4–1.8 $\times$ and cuSOLVER geqrf by 1.8–13.5 $\times$.

II. OUR MIXED-PRECISION ALGORITHM: MRCQR

MRCQR constructs an upper triangular preconditioner $\mathbf{R}_s$ — the R -factor from the thin QR decomposition of $\Omega \mathbf{A} \in \mathbb{R}^{c \times n}$ with $n \leq c \ll m$ — such that $\kappa_2(\mathbf{A}\mathbf{R}_s^{-1}) = \mathcal{O}(1)$

with high probability [23], [24]. Applying `CholQR` to the preconditioned matrix then yields $\|I - \hat{Q}^\top \hat{Q}\|_2 = \mathcal{O}(u)$ [25].

Algorithm 2 MRCQR: Mixed-precision, Randomized-preconditioned Cholesky-QR

Input: $A \in \mathbb{R}^{m \times n}$ with $m \gg n$ and $\text{rank}(A) = n$;
 sketch size c (default $c = 3n$ [25]); sketch precision
 $p \in \{\text{half}, \text{single}, \text{double}\}$

Output: $Q \in \mathbb{R}^{m \times n}$ and upper triangular $R \in \mathbb{R}^{n \times n}$

Stage 1: Compute preconditioner in precision p

1: Draw random Rademacher diagonal $D \in \{+1, -1\}^m$ and
 row indices $\mathcal{S} \subset \{1, \dots, \lfloor m/2 \rfloor + 1\}$, $|\mathcal{S}| = c$

▷ Only $\lfloor m/2 \rfloor + 1$ unique bins due to Hermitian
 symmetry of real FFT

2: **[DiagMult]** $\tilde{A} \leftarrow \text{fl}_p(D \cdot A)$

▷ Sign flip + cast A to precision p

3: **[FFT]** $\hat{A} \leftarrow \text{FFT}(\tilde{A})$

▷ Column-wise real-to-complex FFT in precision p

4: **[Select]** $A_s \leftarrow \sqrt{c/m} \cdot \text{Re}(\hat{A})[\mathcal{S}, :]$

▷ Sample c rows; take real parts

5: **[SketchQR]** $[Q_s, R_s] \leftarrow \text{thinQR}(A_s)$

▷ QR in precision p

6: Promote R_s to double precision

Stage 2: CholQR in double precision

7: $A_1 \leftarrow AR_s^{-1}$ ▷ A_1 is well-conditioned

8: $[Q, R_2] \leftarrow \text{CholQR}(A_1)$ ▷ Algorithm 1

Stage 3: Recover R

9: $R \leftarrow R_2 R_s$ ▷ Upper-triangular factor of A

A. Algorithm Description

MRCQR uses the SRTT [23] as the sketching operator $\Omega = SFD$, where D is a diagonal Rademacher matrix, F applies column-wise FFTs, and S subsamples c rows. We now explain each implementation choice.

Applying D requires no floating-point multiplications: since $d_i \in \{+1, -1\}$, the sign flip reduces to XOR-ing the IEEE-754 sign bit (the MSB: bit 63 for FP64, bit 31 for FP32, bit 15 for FP16, using IEEE-754 numbering from the most significant end) and is fused with the precision cast in a single pass over A . The diagonal multiply achieves modest speedups: since A must always be read in its original double precision (8 bytes per element) regardless of the output precision, the read traffic is fixed and the bandwidth savings are fundamentally limited.

We use the FFT rather than the Walsh–Hadamard Transform (WHT), despite WHT requiring no floating-point multiplications. Our benchmarks show WHT is $3\times$ slower than `cuFFT` at $m = 2^{20}$, an advantage we attribute to `cuFFT`'s engineering maturity rather than any algorithmic difference. Since A is

real-valued, its m -point DFT satisfies the Hermitian symmetry $\hat{A}[m-k] = \overline{\hat{A}[k]}$ for $k = 1, \dots, \lfloor m/2 \rfloor$, so only $\lfloor m/2 \rfloor + 1$ unique frequency bins exist per column. We select c bins and collect their real parts, forming a $c \times n$ real sketch S with $S_{ij} = \text{Re}(\hat{A}[k_i, j])$. An alternative is to retain both real and imaginary parts, yielding a $2c \times n$ sketch at no extra FFT cost but doubling the sketch QR cost. Our experiments show that the Re-only variant achieves double-precision orthogonality $\|\hat{Q}^\top \hat{Q} - I\|_2 \approx u$ across all tested condition numbers, while halving the sketch QR cost. We use $c = 3n$ samples, giving a $3\times$ overdetermined sketch; this choice is based on extensive empirical experiments in [25], which found it to reliably produces a well-conditioned preconditioner R_s in practice.

For the small n values considered here, sketch QR accounts for a small fraction of total sketch time. Nevertheless, two limitations are worth noting. First, `Sgeqrf` achieves only $\sim 1.09\times$ speedup over `Dgeqrf`: `cuSOLVER`'s blocked Householder QR interleaves a sequential BLAS-2 panel factorization that costs the same regardless of precision, creating an Amdahl bottleneck. Second, `cuSOLVER` provides no half-precision `geqrf`; the half-precision sketch variant therefore promotes the sketch matrix to single precision and executes the same `Sgeqrf` call, giving identical `SketchQR` times for both precision levels.

In Stage 2, R_s is promoted to double precision before the triangular solve $A_1 \leftarrow AR_s^{-1}$: performing this solve in reduced precision would incur a backward error proportional to u_p on the full $m \times n$ matrix, degrading the orthogonality achieved by the subsequent double-precision `CholQR`. Within `CholQR` we use `dgemm` rather than `dsyrk` for the Gram matrix: profiling shows `dsyrk` significantly underperforms `dgemm` for tall-and-skinny shapes on the Nvidia H100 GPU. Finally, the triangular solve uses `dtrsm` directly rather than explicitly forming R_s^{-1} and applying `dgemm`: explicitly forming R_s^{-1} squares the condition number in the error bound, potentially losing all significant digits when $\kappa_2(R_s)$ is non-trivial.

B. Mixed-Precision Justification

The SRTT sketching operator ensures that $\kappa_2(AR_s^{-1}) = \mathcal{O}(1)$ with high probability [23], [24]. Here $A_1 \equiv AR_s^{-1}$ is the ideal preconditioned matrix in exact arithmetic. In finite arithmetic, computing R_s in reduced precision produces a perturbed factor $R_s + E$, so the actual computed object is $\hat{A}_1 \equiv A(R_s + E)^{-1}$. Theorem 1 shows that $\kappa_2(\hat{A}_1)$ remains $\mathcal{O}(1)$ provided the relative error $\varepsilon \equiv \|E\|_2 / \|R_s\|_2$ from computing R_s in reduced precision satisfies $\varepsilon < \kappa_2(R_s)^{-1}$.

A standard floating-point analysis gives $\varepsilon = \mathcal{O}(u_p)$ for computing R_s in precision u_p . Since $\kappa_2(R_s) \approx \kappa_2(A)$, the condition $\varepsilon < \kappa_2(R_s)^{-1}$ becomes $u_p \cdot \kappa_2(A) \lesssim 1$, directly yielding the precision thresholds in Algorithm 2: single precision ($u_s \approx 10^{-8}$) is safe when $\kappa_2(A) \lesssim 10^8$, and half precision ($u_h \approx 10^{-4}$) when $\kappa_2(A) \lesssim 10^4$. Once $\hat{A}_1$ is well-conditioned, the analysis of [25, Corollary 3.2] guarantees that applying `CholQR` to $\hat{A}_1$ achieves $\|I - \hat{Q}^\top \hat{Q}\|_2 = \mathcal{O}(u)$,

where $\widehat{\mathbf{Q}}$ is the output of Algorithm 2, completing the theoretical justification for MRCQR.

We now state this precisely.

Theorem 1. *Assume $\mathbf{A}$ has full column rank and $\mathbf{R}_s$ is nonsingular. Let $\mathbf{A}_1 \equiv \mathbf{A}\mathbf{R}_s^{-1}$, $\widehat{\mathbf{A}}_1 \equiv \mathbf{A}(\mathbf{R}_s + \mathbf{E})^{-1}$, and $\varepsilon \equiv \frac{\|\mathbf{E}\|_2}{\|\mathbf{R}_s\|_2}$. If $\|\mathbf{E}\|_2\|\mathbf{R}_s^{-1}\|_2 < 1$, then*

$$\kappa_2(\widehat{\mathbf{A}}_1) \leq \kappa_2(\mathbf{A}_1) \frac{1 + \varepsilon \kappa_2(\mathbf{R}_s)}{1 - \varepsilon \kappa_2(\mathbf{R}_s)}.$$

The assumption $\|\mathbf{E}\|_2\|\mathbf{R}_s^{-1}\|_2 < 1$ holds whenever $\kappa_2(\mathbf{A}) \lesssim \mathbf{u}^{-1} \approx 10^{16}$, as confirmed by our experiments.

Proof. Since $\|\mathbf{E}\|_2\|\mathbf{R}_s^{-1}\|_2 < 1$, the Banach lemma [5, Lemma 2.3.3] implies that $\mathbf{R}_s + \mathbf{E}$ is nonsingular, so that $\widehat{\mathbf{A}}_1$ is well defined. To relate $\widehat{\mathbf{A}}_1$ to $\mathbf{A}_1$, factor out $\mathbf{R}_s^{-1}$,

$$\widehat{\mathbf{A}}_1 = \mathbf{A}\mathbf{R}_s^{-1}(\mathbf{I} + \mathbf{E}\mathbf{R}_s^{-1})^{-1} = \mathbf{A}_1(\mathbf{I} + \mathbf{E}\mathbf{R}_s^{-1})^{-1}.$$

The Banach lemma also implies

$$\|\widehat{\mathbf{A}}_1\|_2 \leq \frac{\|\mathbf{A}_1\|_2}{1 - \|\mathbf{E}\|_2\|\mathbf{R}_s^{-1}\|_2} = \frac{\|\mathbf{A}_1\|_2}{1 - \varepsilon \kappa_2(\mathbf{R}_s)}.$$

From $\widehat{\mathbf{A}}_1$ having full column rank follows that its left inverse equals

$$\widehat{\mathbf{A}}_1^\dagger = (\mathbf{I} + \mathbf{E}\mathbf{R}_s^{-1})\mathbf{A}_1^\dagger.$$

Analogous to the above, we bound

$$\begin{aligned} \|\widehat{\mathbf{A}}_1^\dagger\|_2 &\leq \|\mathbf{A}_1^\dagger\|_2 (1 + \|\mathbf{E}\|_2\|\mathbf{R}_s^{-1}\|_2) \\ &= \|\mathbf{A}_1^\dagger\|_2 (1 + \varepsilon \kappa_2(\mathbf{R}_s)). \end{aligned}$$

Combining gives

$$\kappa_2(\widehat{\mathbf{A}}_1) \leq \kappa_2(\mathbf{A}_1) \frac{1 + \varepsilon \kappa_2(\mathbf{R}_s)}{1 - \varepsilon \kappa_2(\mathbf{R}_s)}.$$

□

C. Adaptive Precision Selection

The sketch precision p is the only user-supplied parameter in MRCQR, and only a *coarse* classification of $\kappa_2(\mathbf{A})$ into one of three regimes is required to set it: $\kappa_2(\mathbf{A}) < 10^4$ (use half), $10^4 \leq \kappa_2(\mathbf{A}) < 10^8$ (use single), or $\kappa_2(\mathbf{A}) \geq 10^8$ (use double). The precise value of $\kappa_2(\mathbf{A})$ is never needed. In many practical settings this regime is known from prior knowledge of the problem—for instance, from the mesh size in a discretized PDE or the spectrum of a covariance matrix—and no estimation is required (see Fig. 1 for the effect of an incorrect choice).

When the regime is not known a priori, several options are available. The cheapest is a *CholQR probe*: apply Algorithm 1 to $\mathbf{A}$ and invert the orthogonality deviation $q \equiv \|\mathbf{I} - \widehat{\mathbf{Q}}^\top \widehat{\mathbf{Q}}\|_2$ using the bound

$$\kappa_2(\mathbf{A}) \geq \sqrt{\frac{q}{\mathbf{u}(1+q)}}, \quad (2)$$

derived from [25, Theorem 2.2]. This probe costs one `CholQR` call and is essentially free when `CholQR` was already the intended first attempt. Alternatively, the block 1-norm estimator of [26] applied to $\mathbf{A}^\top \mathbf{A}$ classifies the regime at $\mathcal{O}(n^2)$ cost, exploiting the bound $\kappa_2(\mathbf{A})^2 \leq n\|\mathbf{A}^\top \mathbf{A}\|_1\|(\mathbf{A}^\top \mathbf{A})^{-1}\|_1$ [27, Exercise 2.6]. Machine-learning-based estimators [28] offer a further avenue when a training distribution is available. A full implementation of condition number estimation is beyond the scope of this work; our experiments fix $\kappa_2(\mathbf{A})$ at construction time. Developing an integrated, GPU-friendly estimator is a natural direction for future work.

When QR is performed *repeatedly* on a sequence of similar matrices—such as the block Arnoldi process [3], randomized SVD via power iteration [4], or time-stepping schemes—the conditioning regime typically changes slowly or predictably. A single regime classification then applies to many subsequent calls, amortizing its cost to a negligible fraction of the total computation.

If the precision is underestimated, the deviation from orthonormality will exceed $\mathcal{O}(\mathbf{u})$, which is immediately detectable by checking $\|\mathbf{I} - \widehat{\mathbf{Q}}^\top \widehat{\mathbf{Q}}\|_2$. The remedy is to rerun MRCQR at the next higher precision level. This self-correcting property means that an aggressive (low-precision) initial guess carries no risk beyond one wasted factorization.

III. NUMERICAL EXPERIMENTS

A. Experimental Setup

MRCQR is implemented in C++/CUDA using vendor-optimized libraries: cuBLAS for GEMM and triangular solves, cuSOLVER for QR and Cholesky factorizations, and cuFFT for column-wise FFTs. Half-precision FFTs use the `cufftXt` API (CUDA_R_16F input, CUDA_C_16F output), since the standard `cufftExecR2C` does not expose a half-precision interface. All results were obtained with CUDA 13.1 and GCC 11.5.0 on an NVIDIA H100 PCIe (80 GB VRAM); reported times exclude host-device transfers and are averaged over 5 trials.

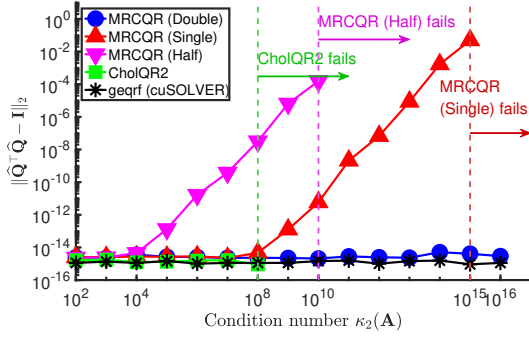
We compare against three baselines: (1) cuSOLVER `geqrf`, the standard industry-stable GPU QR baseline; (2) `CholQR2` [12], implemented by calling our `CholQR` routine twice, included for stability comparison; and (3) `rand-cholQR` [18], a state-of-the-art randomized preconditioned `CholQR` using a two-level sparse-plus-dense multitasket, provided by the authors and built with Kokkos 5.1.1 [29].

Our implementation of MRCQR can be found at <https://github.com/as1805/MRCQR>.

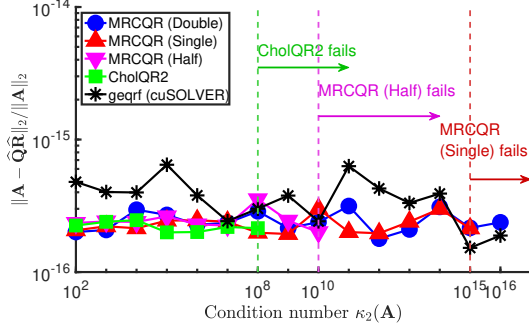
B. Numerical Stability

We assess the orthogonality and backward stability of MRCQR against `geqrf` (the stable reference) and `CholQR2` (known to break down for $\kappa_2(\mathbf{A}) \gtrsim 10^8$ [12]) across a wide range of condition numbers. `rand-cholQR` is omitted here as its stability is established in [18].

We construct test matrices $\mathbf{A} \in \mathbb{R}^{m \times n}$ with prescribed condition numbers via $\mathbf{A} = \mathbf{U}\Sigma\mathbf{V}^\top$, where $\mathbf{U}$, $\mathbf{V}$ have



(a) Deviation from orthonormality $\|\hat{Q}^\top \hat{Q} - I\|_2$ vs. $\kappa_2(\mathbf{A})$



(b) Relative residual $\|\mathbf{A} - \hat{Q}\hat{R}\|_2 / \|\mathbf{A}\|_2$ vs. $\kappa_2(\mathbf{A})$

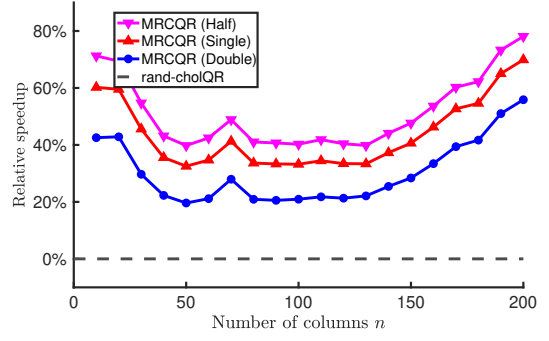
Fig. 1: CholQR2 breaks down past $\kappa_2(\mathbf{A}) = 10^8$ (Gram matrix loses positive definiteness). MRCQR (half-precision sketch) breaks down past $\kappa_2(\mathbf{A}) = 10^{10}$, and MRCQR (single-precision sketch) breaks down past $\kappa_2(\mathbf{A}) = 10^{15}$ (underflow of the smallest singular value of $\mathbf{A}$).

orthonormal columns (obtained by QR of random matrices with entries uniform on $[-1, 1]$) and the singular values are geometric:

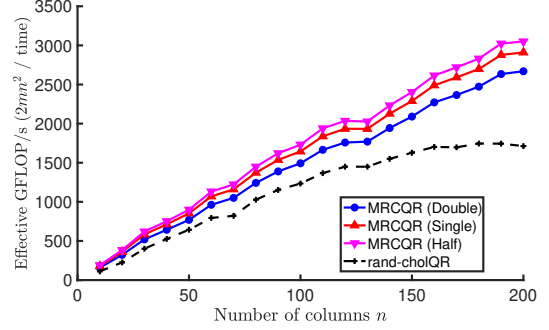
$$\sigma_i = \kappa^{1/2 - i/(n-1)}, \quad i = 0, \dots, n-1, \quad (3)$$

giving $\kappa_2(\mathbf{A}) = \kappa$ exactly. We fix $m = 2^{17}$, $n = 50$, and sweep $\kappa \in \{10^2, 10^3, \dots, 10^{16}\}$.

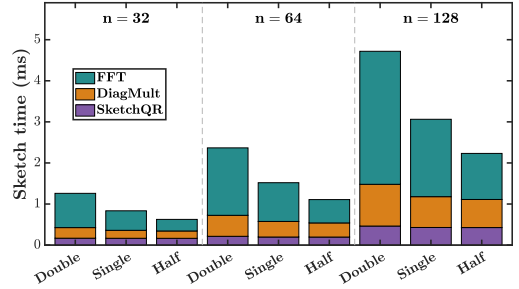
Figure 1 shows deviation from orthonormality and relative residual for all methods. `geqrf` achieves the gold standard of $\|I - \hat{Q}^\top \hat{Q}\|_2 \approx 10^{-15}$ for all κ ; MRCQR (double-precision sketch) matches this at $\approx 5 \times 10^{-15}$ across all tested condition numbers. Confirming the thresholds of Theorem 1, MRCQR (single-precision sketch) maintains $\mathcal{O}(\mathbf{u})$ orthogonality for $\kappa_2(\mathbf{A}) \lesssim 10^8$ and MRCQR (half-precision sketch) for $\kappa_2(\mathbf{A}) \lesssim 10^4$; beyond these thresholds orthogonality degrades. CholQR2 breaks down past $\kappa_2(\mathbf{A}) = 10^8$, consistent with known analysis [12]. Notably, MRCQR (single-precision sketch) continues to produce a result beyond 10^8 —albeit with degraded orthogonality—whereas CholQR2 fails entirely. This graceful degradation is useful in practice: a user who underestimates $\kappa_2(\mathbf{A})$ receives a detectable warning rather than a hard failure. All methods that do not break down are backward stable (Fig. 1b).



(a) Speedup of MRCQR over `rand-cholQR` vs. n



(b) GFlop/s of MRCQR and `rand-cholQR` vs. n



(c) Sketch-phase timing breakdown across three precisions

Fig. 2: Performance of MRCQR vs. `rand-cholQR` [18] on NVIDIA H100 GPU, $m = 2^{20}$, averaged over 5 trials.

C. Runtime Performance

We compare MRCQR (FP64, FP32, FP16 sketch) against `rand-cholQR` [18] and `cuSOLVER geqrf`, fixing $m = 2^{20}$ and varying n . All three methods are stable for $\kappa_2(\mathbf{A}) \lesssim 10^{16}$, making this a fair speed comparison. The sketch size for `rand-cholQR` follows the authors' recommendation [18, Section 6.2]: $p_1 = \lceil 8.24(n^2 + n) \rceil$ (CountSketch) and $p_2 = \lceil 74.3 \log(p_1) \rceil$ (Gaussian); note p_1 grows quadratically in n , whereas MRCQR uses $c = 3n$. Although the CountSketch is a sparse transform and is theoretically efficient, realising this on GPU requires a highly optimized sparse kernel; the Kokkos [29] implementation provided by the authors may not reflect the full performance potential of `rand-cholQR`.

Figure 2 illustrates three complementary aspects of MRCQR's performance. Figure 2a shows that all three precision variants outperform `rand-cholQR`, with the gap increasing

TABLE I: Wall-clock time (ms) on H100, $m = 2^{20}$, 5 trials. Speedup = rand-choQR or geqrf divided by MRCQR (FP16).

n	MRCQR			rand-choQR	geqrf	Speedup vs. MRCQR (FP16)	
	FP64	FP32	FP16			vs rand-choQR	vs geqrf
20	2.60	2.33	2.19	3.71	29.6	1.69×	13.5×
40	5.22	4.71	4.46	6.38	16.6	1.43×	3.7×
60	7.84	7.06	6.67	9.50	20.9	1.42×	3.1×
80	10.80	9.77	9.26	13.06	17.4	1.41×	1.9×
100	14.05	12.75	12.11	16.99	23.0	1.40×	1.9×
120	17.18	15.62	14.84	20.84	28.2	1.40×	1.9×
140	21.15	19.32	18.41	26.53	36.8	1.44×	2.0×
160	23.64	21.56	20.53	31.54	41.0	1.54×	2.0×
180	27.48	25.18	24.00	38.94	53.5	1.62×	2.2×
200	31.43	28.83	27.50	48.98	50.5	1.78×	1.8×

TABLE II: Sketch sub-operation timings (ms) at $n = 64$, $m = 2^{20}$. Ratios across precisions are approximately n -independent.

Precision	DiagMult	FFT	SketchQR	Total
FP64	0.511	1.642	0.214	2.367
FP32	0.379	0.943	0.197	1.518
FP16	0.344	0.568	0.196	1.107
FP64/FP32	1.35×	1.74×	1.09×	1.56×
FP32/FP16	1.10×	1.66×	1.01×	1.37×

with n ; lower sketch precision yields greater speedup, confirming the practical recommendation: use the lowest precision consistent with $\kappa_2(\mathbf{A})$ —half for $\kappa_2(\mathbf{A}) \lesssim 10^4$, single for $\kappa_2(\mathbf{A}) \lesssim 10^8$, and double otherwise. Figure 2b shows the effective flop rates of MRCQR and rand-choQR: MRCQR achieves substantially higher utilization, reflecting the benefit of building on highly optimized vendor libraries (cuFFT, cuBLAS) rather than custom sparse kernels. Figure 2c shows the sketch-phase timing breakdown versus n for all three precisions; the ratios between precision tiers remain approximately constant across n . The speedup from lower precision is driven primarily by the FFT, which dominates the sketch cost and achieves $\sim 1.7\times$ per precision level.

Table I compares MRCQR (three precisions), rand-choQR, and cuSOLVER’s geqrf on the H100. Three observations stand out. First, MRCQR (FP16) is 1.4–1.8 $\times$ faster than the provided rand-choQR implementation across all tested n , with the gap widening as n grows. This reflects not algorithmic superiority but the ease of implementing MRCQR efficiently: its SRTT-based sketch maps directly onto highly optimized vendor libraries (cuFFT), requiring no custom sparse GPU kernel. Second, the speedup of MRCQR (FP16) over geqrf is 13.5 $\times$ at $n = 20$: geqrf incurs a large fixed kernel launch and synchronization overhead that dominates for small n , while MRCQR’s SRTT-based sketch avoids this entirely. At larger n (80–200), where both methods are dominated by $\mathcal{O}(mn^2)$ work, the speedup stabilizes at 1.8–2.2 $\times$, reflecting MRCQR’s BLAS-3 efficiency advantage. Third, the FP16 sketch is $\approx 2\times$ faster than FP64 at no accuracy cost (for $\kappa_2(\mathbf{A}) \lesssim 10^4$), confirming the mixed-precision benefit quantified in Table II.

Table II breaks down sketch-phase timings at $n = 64$; the per-precision speedup ratios are approximately n -independent.

TABLE III: Cost breakdown of MRCQR at $m = 2^{20}$, 10 trials (ms). TRSM $\times 2$ = TrsmPre + TrsmFin; GEMM = Gram matrix; Sketch = full SRTT phase. TRSM and GEMM costs are identical across precisions; only Sketch changes.

n	TRSM $\times 2$	GEMM	Sk. FP64	Sk. FP32	Sk. FP16
16	0.93	0.12	0.62	0.40	0.29
32	2.05	0.17	1.26	0.84	0.63
64	5.00	0.29	2.37	1.52	1.11
128	11.65	0.68	4.72	3.06	2.23

The FFT dominates sketch cost and achieves $\sim 1.7\times$ speedup per precision level. DiagMult achieves more modest gains ($\sim 1.35\times$ FP64 $\rightarrow$ FP32, $\sim 1.10\times$ FP32 $\rightarrow$ FP16) because $\mathbf{A}$ must always be read in FP64, limiting bandwidth savings. SketchQR achieves only $\sim 1.09\times$ speedup from FP64 to FP32 and no further gain from FP32 to FP16, for the reasons discussed in Section II-A.

Table III reveals the cost structure of MRCQR. Three points stand out. First, the two triangular solves (TRSM) dominate the runtime: TRSM accounts for 56% of total time at $n = 16$, growing to 74% at $n = 128$. This is perhaps surprising given that TRSM is a BLAS-3 operation, but for tall-and-skinny matrices it is memory-bandwidth bound—the sequential wave-front dependency limits GPU parallelism—giving near-linear scaling in n . Second, GEMM is negligible at these column counts (0.12–0.68 ms), confirming that the compute-intensive Gram matrix formation is not the bottleneck in this regime. Third, the Sketch cost is the only quantity that changes with precision: FP16 reduces sketch cost by $\approx 2\times$ over FP64 at each n , while TRSM and GEMM remain unchanged. This directly confirms that mixed-precision sketching targets exactly the right part of the algorithm.

IV. CONCLUSIONS

MRCQR is a numerically stable GPU algorithm for the thin QR factorization of tall-and-skinny matrices, achieving $\|\mathbf{I} - \hat{\mathbf{Q}}\hat{\mathbf{Q}}^\top\|_2 = \mathcal{O}(\mathbf{u})$ for $\kappa_2(\mathbf{A})$ up to 10^{16} —far beyond the 10^8 stability limit of CholQR2. The central insight is that the SRTT-based preconditioner only needs to reduce the effective condition number to $\mathcal{O}(1)$, not achieve full double-precision accuracy: FP32 suffices when $\kappa_2(\mathbf{A}) \lesssim 10^8$ and FP16 when $\kappa_2(\mathbf{A}) \lesssim 10^4$, exactly matching the predicted thresholds. On an NVIDIA H100, MRCQR (FP16) is 1.8–13.5 $\times$ faster than cuSOLVER geqrf and 1.4–1.8 $\times$ faster than rand-choQR, with no custom GPU kernels. Profiling reveals that TRSM dominates at 56–74% of total runtime while the sketch accounts for only 10–20%, so tensor-core-based TRSM reformulation [30] is the most promising direction for further speedup. Precision selection is easily determined from problem structure or a cheap CholQR probe, making MRCQR practical in production HPC workflows.

REFERENCES

- [1] P. G. Constantine and D. F. Gleich, “Tall and skinny QR factorizations in MapReduce architectures,” in *Proceedings of the second international workshop on MapReduce and its applications*, 2011, pp. 43–50.

- [2] r. Björck, *Numerical Methods for Least Squares Problems*. Society for Industrial and Applied Mathematics, 1996.
- [3] Y. Saad, *Iterative Methods for Sparse Linear Systems*, 2nd ed. Society for Industrial and Applied Mathematics, 2003.
- [4] N. Halko, P.-G. Martinsson, and J. A. Tropp, “Finding structure with randomness: Probabilistic algorithms for constructing approximate matrix decompositions,” *SIAM Rev.*, vol. 53, no. 2, pp. 217–288, 2011.
- [5] G. H. Golub and C. F. Van Loan, *Matrix computations*, 4th ed., ser. Johns Hopkins Studies in the Mathematical Sciences. Johns Hopkins University Press, Baltimore, MD, 2013.
- [6] NVIDIA Corporation, “cuSOLVER library documentation,” <https://docs.nvidia.com/cuda/cusolver>, 2024, accessed: 2026-06-03.
- [7] J. Demmel, L. Grigori, M. Hoemmen, and J. Langou, “Communication-optimal parallel and sequential QR and LU factorizations,” *SIAM Journal on Scientific Computing*, vol. 34, no. 1, pp. A206–A239, 2012.
- [8] T. Fukaya, Y. Nakatsukasa, Y. Yanagisawa, and Y. Yamamoto, “CholeskyQR2: a simple and communication-avoiding algorithm for computing a tall-skinny QR factorization on a large-scale parallel system,” in *2014 5th Workshop on Latest Advances in Scalable Algorithms for Large-Scale Systems*, 2014, pp. 31–38.
- [9] J. Thies and M. Röhrig-Zöllner, “Implementation of QR factorization of tall and very skinny matrices on current GPUs,” 2026. [Online]. Available: <https://arxiv.org/abs/2603.20889>
- [10] A. Stathopoulos and K. Wu, “A block orthogonalization procedure with constant synchronization requirements,” *SIAM J. Sci. Comput.*, vol. 23, no. 6, pp. 2165–2182, 2002. [Online]. Available: <https://doi.org/10.1137/S1064827500370883>
- [11] I. Yamazaki, S. Tomov, and J. Dongarra, “Mixed-precision Cholesky QR factorization and its case studies on multicore CPU with multiple GPUs,” *SIAM J. Sci. Comput.*, vol. 37, no. 3, pp. C307–C330, 2015. [Online]. Available: <https://doi.org/10.1137/14M0973773>
- [12] Y. Yamamoto, Y. Nakatsukasa, Y. Yanagisawa, and T. Fukaya, “Roundoff error analysis of the CholeskyQR2 algorithm,” *Electron. Trans. Numer. Anal.*, vol. 44, pp. 306–326, 2015.
- [13] I. S. 754-2019, “IEEE Standard for Floating-Point Arithmetic,” (*Revision of IEEE 754-2008*), pp. 1–84, 2019.
- [14] N. J. Higham, *Accuracy and stability of numerical algorithms*, 2nd ed. Society for Industrial and Applied Mathematics (SIAM), Philadelphia, PA, 2002. [Online]. Available: <https://doi.org/10.1137/1.9780898718027>
- [15] T. Fukaya, R. Kannan, Y. Nakatsukasa, Y. Yamamoto, and Y. Yanagisawa, “Shifted Cholesky QR for computing the QR factorization of ill-conditioned matrices,” *SIAM J. Sci. Comput.*, vol. 42, no. 1, pp. A477–A503, 2020. [Online]. Available: <https://doi.org/10.1137/18M1218212>
- [16] Y. Fan, Y. Guo, and T. Lin, “A novel randomized XR-based preconditioned CholeskyQR algorithm,” 2021, arXiv:2111.11148v2.
- [17] O. Balabanov, “Randomized Cholesky QR factorizations,” 2022, arXiv:2210.09953v2.
- [18] A. J. Higgins, D. B. Szyld, E. G. Boman, and I. Yamazaki, “Analysis of randomized Householder-Cholesky QR factorization with multisketching,” *Numerische Mathematik*, 2025.
- [19] M. Melnichenko, O. Balabanov, R. Murray, J. Demmel, M. W. Mahoney, and P. Luszczek, “CholeskyQR with randomization and pivoting for tall matrices (CQRRT),” *SIAM Journal on Matrix Analysis and Applications*, 2025.
- [20] V. Georgiou, C. Boutsikas, P. Drineas, and H. Anzt, “A mixed precision randomized preconditioner for the LSQR solver on GPUs,” in *High Performance Computing. ISC High Performance 2023*. Springer Nature Switzerland, 2023.
- [21] E. Carson and I. Daužickaitė, “Mixed precision sketching for least-squares problems and its application in GMRES-based iterative refinement,” *SIAM J. Matrix Anal. Appl.*, vol. 46, no. 3, pp. 2041–2060, 2025.
- [22] V. Rokhlin and M. Tygert, “A fast randomized algorithm for overdetermined linear least-squares regression,” *Proc. Natl. Acad. Sci. USA*, vol. 105, no. 36, pp. 13 212–13 217, 2008.
- [23] H. Avron, P. Maymounkov, and S. Toledo, “Blendenpik: supercharging LAPACK’s least-squares solver,” *SIAM J. Sci. Comput.*, vol. 32, no. 3, pp. 1217–1236, 2010.
- [24] I. C. F. Ipsen and T. Wentworth, “The effect of coherence on sampling from matrices with orthonormal columns, and preconditioned least squares problems,” *SIAM J. Matrix Anal. Appl.*, vol. 35, no. 4, pp. 1490–1520, 2014.
- [25] J. E. Garrison and I. C. F. Ipsen, “A randomized preconditioned Cholesky-QR algorithm,” 2024, arXiv:2406.11751.
- Under revision for SIAM J. Matrix Anal. Appl. [Online]. Available: <https://arxiv.org/abs/2406.11751>
- [26] N. J. Higham and F. Tisseur, “A block algorithm for matrix 1-norm estimation, with an application to 1-norm pseudospectra,” *SIAM Journal on Matrix Analysis and Applications*, vol. 21, no. 4, pp. 1185–1201, 2000. [Online]. Available: <https://doi.org/10.1137/S0895479899356080>
- [27] I. C. F. Ipsen, *Numerical matrix analysis*. Society for Industrial and Applied Mathematics (SIAM), Philadelphia, PA, 2009, linear systems and least squares. [Online]. Available: <https://doi.org/10.1137/1.9780898717686>
- [28] E. Carson and X. Chen, “Estimating condition number with graph neural networks,” 2025, arXiv:2603.10277.
- [29] C. R. Trott, D. Lebrun-Grandie, D. Arndt, J. Ciesko, V. Dang, N. Ellingwood, R. Luo, E. McCain, D. Poliakoff, A. Powell, S. Rajamanickam, M. Simberg, D. Turman, and B. Videau, “Kokkos 3: Programming model extensions for the exascale era,” *IEEE Trans. Parallel Distrib. Syst.*, vol. 33, no. 4, pp. 805–817, 2022.
- [30] V. Carrica, M. Onyango, R. Alomairy, E. Ringoot, J. Schloss, and A. Edelman, “Toward portable GPU performance: Julia recursive implementation of TRMM and TRSM,” *arXiv preprint arXiv:2504.13821*, 2025.